TRACTABLE PROBLEMS

Microkernels and Hypervisors

Multi-tenant GPU isolation today relies on hypervisors.
While hypervisors started small, broadly they have grown into full cloud-operating systems. KVM's trusted computing base is roughly ten million lines of code; and while Xen is smaller (roughly 400k LoC) it is still quite large. Both have had VM-escape vulnerabilities — Google Project Zero's 2021 KVM breakout via AMD SVM nested virtualization [1] is a representative example — and the PCI passthrough path that GPU workloads require widens the attack surface further, since IOMMU misconfigurations or missing PCI Access Control Services (ACS) can allow peer-to-peer DMA between devices assigned to different tenants. NVIDIA's Multi-Instance GPU (MIG) partitioning provides hardware-level memory isolation within a single GPU, but the partitioning is managed by the host driver, which runs inside the hypervisor's TCB.

Several projects have aimed to establish trust through verification at the hypervisor (and OS) level. To keep verification tractable, these projects aim to partition the system into a minimal trusted core (which holds exclusive access to page tables and privileged resources, and places policy decisions in a de-privileged layer. This architecture reflects the principles of the microkernel philosophy and many of these systems are microkernels, or draw inspiration from that line of work.

[seL4](https://sel4.systems/) is the poster-child for micro-kernel verification. Popularized in the [DARPA HACMS program](https://www.darpa.mil/research/programs/high-assurance-cyber-military-systems), seL4's proof connents a a top-level specification written in Isabelle to a binary-level semantics of the underlying architecture. The depth of this proof allows it to remove trust from the C compiler (which is known to have bugs) through translation validation.
seL4 supports a range of platforms and comes in different variants for different levels of isolation, including an [MCS mode built for mixed criticality systems](https://docs.sel4.systems/releases/sel4/9.0.0-mcs.html).
While seL4's verification is deep, its proof for modern, server-class machines is somewhat limited.
The proof is fundamentally sequential, meaning that the proof itself does not cover running seL4 across multiple cores. Recent work aims to address this through a multi-kernel configuration (where a separate version of seL4 runs on each core).
In addition the sequential nature of the proof also assumes the absence of DMA by external devices, though practically speaking the underlying assumption really only requires the lack of DMA to seL4-controlled memory.

Sharing the microkernel-philosophy of seL4, the [NOVA microkernel](https://hypervisor.org/) aims to be a modern contender in the verified hypervisor space. In contrast to seL4, NOVA is

highly aggressive with respect to concurrency and modern hardware adoption including features such as encrypted memory, measured boot, as well as IOMMU/System MMU support. While the verification of NOVA is still underway, the proof tackles the problem of concurrent verification from the onset leveraging a highly modular proof strategy and recent work (2026) has begun moving [NOVA's proof to support the weak memory systems](https://bluerocksec.gitlab.io/formal-methods/blogs/2026-03-31-NOVA-relaxed-memory/) present on modern architectures. In contrast to the seL4 proof, NOVA's (partial) proof bottoms out at the source code level leaving the potential for compilers to introduce bugs or vulnerabilities into the final binary.

Amazon's Nitro Isolation Engine (cite) and the seKVM project (cite) share similar goals in bringing verified system software to mainstream environments. Both projects verify a trusted core that runs in hypervisor mode and are driven by unverified but deprivileged components. Unlike seL4 and NOVA, these projects are not microkernels and their APIs are highly tailored to operate within their particular stacks. This narrower interface makes concurrency reasoning more tractable and both proofs support concurrent behaviors.

Beyond Hypervisor Mode

The architecture of isolating a small trustworthy (through verification) core from the higher-level requirements of modern hypervisors produces a good verification story, but the expressiveness of the API to the trusted module introduces a trade-off between capabilities and security. Projects such as seKVM embed policies within the verified core ensuring, e.g. that pages can not be shared (or read/write shared) between different domains. Embedding policies such as this directly within the verified code provides a strong guarantee, but can be limiting when trying to leverage these systems in contexts that seek more flexibility, e.g. allowing multiple domains the ability to communicate via shared memory.

On the microkernel side, seL4 and NOVA provide rich capability-based APIs that enable user-mode applications significant flexibility in resource allocation; however, the cost of this is that security guarantees such as isolation rely fundamentally on the correctness of the code that sets up these policies. seL4 provides the [CAmkES framework](https://docs.sel4.systems/projects/camkes/) for configuring resources, but only supports static configurations. NOVA's idiomatic usage is intended to be more dynamic; however, establishing isolation properties in this dynamic world requires user-mode verification. Efforts exist on both of these platforms to establish properties at the user-mode level. In the seL4 ecosystem, [Kry10](https://kry10.com) aims to raise the foundational guarantees of seL4 up the stack, and BlueRock Security has described how it has combined hypervisor- and user-mode verifications to build [proofs of complets systems](https://dl.acm.org/doi/10.1145/3713082.3730387) (and technical reports: [VMM](https://bluerocksec.gitlab.io/formal-methods/tech_reports/verifying-a-virtual-machine-monitor/) and [vSwitch](https://bluerocksec.gitlab.io/formal-methods/tech_reports/protocol-completion-of-a-robust-c-virtual-switch/)).

Outlook & Next Steps

The "small trustworthy core" has been the crucial enabler for research and industrial systems to attack the hypervisor problem; however, the approach is only goes so far. Neither seL4 nor NOVA, in isolation, is capable of running virtual machines, let alone supporting advanced features that many users expect from modern hypervisors such as dynamic provisioning and virtual machine migration. Brining these core components into a trustworthy foundational stack is possible, but requires additional engineering effort to verify user-mode applications running atop these components. A core roadblock in this space is the sheer size of the technology stack that hypervisors support. Identifying a core feature set that would enable a wide swath of applications while still remaining tractable to verify with current technology is a key enabler for beginning to build these ecosystems.

Device Drivers

A notable omission from the previous section is device drivers.
All of the hypervisor projects noted above explicitly remove device drivers from the trusted core, a necessary step in controlling the size of the verified code.
Following the microkernel philosophy, in these systems, device drivers are implemented in user-mode.
While work on [verifying device drivers exists](https://sel4.systems/Summit/2025/abstracts2025.html#a-verified-zynqmp), a major limiting factor is hardware complexity and modeling. While CPUs generally fall into only a handful of architectures,
BlueRock's work on a [verified network switch](https://bluerocksec.gitlab.io/formal-methods/tech_reports/protocol-completion-of-a-robust-c-virtual-switch/) provides some insight into the complexity of reasoning about a VirtIO device, but the lack of source code makes it difficult to fully evaluate.

Device Drivers

(The text below is from the original document and is really good, but it is much more focused on device drivers than hypervisors themselves).

seL4, the only general-purpose microkernel with a complete functional-correctness proof, has no GPU driver support at all. Its new device driver framework (sDDF) handles network and block devices, but nothing with the complexity of a modern GPU. NOVA [2] is the other candidate worth naming here: it is open-source (GPL-2) with an open-source proof, and though the

verification is not as far along as seL4′s, the portions that are complete already account for concurrency and weak memory — both of which fundamentally change the verification challenge rather than merely extending it. Closing the GPU gap — giving either kernel a verified GPU driver — is the subject of §.

Even where MIG is deployed, GPU memory is not always scrubbed between context switches. Trail of Bits demonstrated this with LeftoverLocals [3]: on AMD, Apple, and Qualcomm GPUs, a co-resident process could read local memory left behind by a previous kernel, recovering roughly 5.5 MB per GPU invocation — enough to reconstruct an LLM's token-by-token output with high fidelity. NVIDIA hardware was not affected in that specific case, but the deeper issue is architectural. GPUs were designed for throughput, not isolation, and the memory hierarchy reflects that priority. Shared L2 caches, shared interconnects, and performance counters that leak timing information all create side channels across tenant boundaries. The NVBleed attack [4] showed that contention on NVLink interconnects between GPUs lets an attacker fingerprint which deep-learning model a co-tenant is running with 97.8% accuracy, and the attack works across VM boundaries on Google Cloud — even after NVIDIA patched performance-counter access, the timing channel alone still yields F1 scores above 83%.

Confidential VMs compound the problem. CPU-side TEEs like AMD SEV-SNP encrypt guest memory so the hypervisor cannot read it, but extending that guarantee to a GPU is an open engineering problem. SEV-SNP requires the IOMMU in non-passthrough mode to prevent peripherals from reaching encrypted memory, which directly conflicts with the PCIe passthrough that GPU workloads need. AMD's SEV-TIO extension [5] is meant to bridge this gap using PCI-SIG's TDISP protocol, but it is not yet widely deployed, and the attestation story for a GPU behind a TDISP-secured link is still being worked out. In the meantime, any "confidential AI" deployment that claims SEV-SNP protection while using GPU passthrough has a gap in its threat model that the marketing materials do not mention.